

DBMS PL-SQL Lab 1

Name: Swagathchandran S

Roll No.: AM.SC.U4AIE23154

1. Do all the questions discussed in the class

```
CREATE TABLE Dept(  
    dept_no INT PRIMARY KEY,  
    dname VARCHAR(25)  
);
```

```
CREATE TABLE EMP(  
    eno INT PRIMARY KEY,  
    ename VARCHAR(25),  
    sal NUMERIC(10,2),  
    dno INT REFERENCES Dept(dept_no)  
);
```

```
INSERT INTO Dept VALUES (10, 'Sales');  
INSERT INTO Dept VALUES (20, 'Marketing');  
INSERT INTO Dept VALUES (30, 'Research');  
INSERT INTO Dept VALUES (40, 'Finance');
```

```
INSERT INTO EMP VALUES (101, 'Arun', 25000, 10);  
INSERT INTO EMP VALUES (102, 'Balan', 30000, 20);  
INSERT INTO EMP VALUES (103, 'Charan', 22000, 10);  
INSERT INTO EMP VALUES (104, 'Kunju', 45000, 30);  
INSERT INTO EMP VALUES (105, 'Esha', 28000, 40);  
INSERT INTO EMP VALUES (301, 'Isa', 45000.00, 30);  
INSERT INTO EMP VALUES (302, 'Surya', 80000.00, 30);
```

```

-- Function 1: Accepts employee number and returns salary of that employee
CREATE OR REPLACE FUNCTION emp_sal(emp_no EMP.eno%TYPE)
RETURNS EMP.sal%TYPE AS
$$
DECLARE
    salary EMP.sal%TYPE;
BEGIN
    SELECT sal INTO salary FROM EMP WHERE eno = emp_no;
    RETURN salary;
END;
$$
LANGUAGE plpgsql;

```

```

-- Function 3: Takes eno and gives the employee 10% hike in salary
-- if average salary of the corresponding dept is greater than employee's current salary
CREATE OR REPLACE FUNCTION fn1(emp_no EMP.eno%TYPE)
RETURNS VOID AS
$$
DECLARE
    emp_salary EMP.sal%TYPE;
    dept_no EMP.dno%TYPE;
    dept_avg EMP.sal%TYPE;
BEGIN
    -- Get employee salary
    SELECT sal, dno INTO emp_salary, dept_no FROM EMP WHERE eno = emp_no;

    -- Get average salary of the employee's department
    SELECT AVG(sal) INTO dept_avg FROM EMP WHERE dno = dept_no;

    -- If employee earns less than department average, give 10% hike
    IF emp_salary < dept_avg THEN
        UPDATE EMP
        SET sal = sal * 1.10
        WHERE eno = emp_no;
    END IF;
END;
$$
LANGUAGE plpgsql;

```

```
SELECT emp_sal(101);
```

	emp_sal numeric 
1	25000.00

```
SELECT no_of_emp(20);
```

	no_of_emp	
	integer	
1		1

```
SELECT fn1(301);
```

	fn1	
	void	
1		

2. Consider the following schema .

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

a) Write a function ProjectLoad that returns the total project working hours for the given eno.

b)

c) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

.

```
-----2-----
CREATE TABLE Emp(
  eno INT PRIMARY KEY,
  ename VARCHAR(25),
  sal NUMERIC(8,2)
);

CREATE TABLE EMP_PROJ(
  eno INT REFERENCES Emp(eno),
  pno INT,
  prjt_hrs INT,
  PRIMARY KEY(eno, pno)
);

INSERT INTO EMP_PROJ VALUES (101, 1, 15);
INSERT INTO EMP_PROJ VALUES (101, 2, 20);
INSERT INTO EMP_PROJ VALUES (102, 1, 10);
INSERT INTO EMP_PROJ VALUES (102, 3, 18);
INSERT INTO EMP_PROJ VALUES (301, 3, 16);
INSERT INTO EMP_PROJ VALUES (301, 5, 22);
INSERT INTO EMP_PROJ VALUES (302, 2, 35);
INSERT INTO EMP_PROJ VALUES (302, 5, 28);
```

```

-- a) Function ProjectLoad: Returns total project working hours for given eno
CREATE OR REPLACE FUNCTION ProjectLoad(emp_no Emp.eno%TYPE)
RETURNS INT AS
$$
DECLARE
    total_hours INT;
BEGIN
    SELECT COALESCE(SUM(prjt_hrs), 0) INTO total_hours FROM EMP_PROJ WHERE eno = emp_no;
    RETURN total_hours;
END;
$$
LANGUAGE plpgsql;

-- c) PL/SQL code to update salary if employee earns less than average
-- New salary = current sal + (average sal - current sal)
CREATE OR REPLACE FUNCTION update_salary_below_avg(emp_no Emp.eno%TYPE)
RETURNS VOID AS
$$
DECLARE
    current_sal Emp.sal%TYPE; avg_sal Emp.sal%TYPE;
    salary_diff NUMERIC(8,2);
BEGIN
    -- Get current salary
    SELECT sal INTO current_sal FROM Emp WHERE eno = emp_no;

    -- Get average salary
    SELECT AVG(sal) INTO avg_sal FROM Emp;

    -- Check if current salary is less than average
    IF current_sal < avg_sal THEN
        salary_diff := avg_sal - current_sal;
        UPDATE Emp SET sal = sal + salary_diff WHERE eno = emp_no;
        RAISE NOTICE 'Employee % salary updated from % to %', emp_no, current_sal, current_sal + salary_diff;
    ELSE
        RAISE NOTICE 'Employee % salary (%) is already at or above average (%)', emp_no, current_sal, avg_sal;
    END IF;
END;
$$
LANGUAGE plpgsql;

```

Select ProjectLoad(101);

	projectload integer
1	35

Select update_salary_below_avg(301);

	update_salary_below_avg void
1	

3. Consider the following tables
Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
CREATE TABLE Item(  
    ino INT PRIMARY KEY,  
    iname VARCHAR(100),  
    unit_price NUMERIC(8,2)  
);
```

```
CREATE TABLE Transaction(  
    tr_no INT PRIMARY KEY,  
    ino INT REFERENCES Item(ino),  
    qty INT  
);
```

```
INSERT INTO Item VALUES (101, 'Pen',10);  
INSERT INTO Item VALUES (102, 'Notebook',40);  
INSERT INTO Item VALUES (103, 'Marker',25);  
INSERT INTO Item VALUES (104, 'Eraser',5);
```

```
INSERT INTO Transaction VALUES (1, 101, 2);  
INSERT INTO Transaction VALUES (2, 102, 5);  
INSERT INTO Transaction VALUES (3, 102, 3);  
INSERT INTO Transaction VALUES (4, 103, 4);  
INSERT INTO Transaction VALUES (5, 103, 1);  
INSERT INTO Transaction VALUES (6, 103, 2);
```

```
CREATE OR REPLACE FUNCTION delete_low_transaction_item(item_no Item.ino%TYPE)  
RETURNS VARCHAR AS  
$$  
DECLARE  
    transaction_count INT;  
    item_name VARCHAR(100);  
BEGIN  
    SELECT iname INTO item_name FROM Item WHERE ino = item_no;  
  
    IF item_name IS NULL THEN  
        RETURN 'Error: Item number ' || item_no || ' does not exist';  
    END IF;  
  
    SELECT COUNT(*) INTO transaction_count FROM Transaction WHERE ino = item_no;  
    IF transaction_count < 2 THEN  
        DELETE FROM Transaction WHERE ino = item_no;  
        DELETE FROM Item WHERE ino = item_no;  
    END IF;  
END;  
$$  
LANGUAGE plpgsql;
```


4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)

- Write a function that accepts a threshold value and a customer number .
The program updates the c_type based on the threshold value. Ie. If balance > threshold then class A, else class B.
- Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
- Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
CREATE TABLE BRANCHES(  
    br_no INT PRIMARY KEY,  
    br_name VARCHAR(100),  
    loc VARCHAR(100)  
);  
  
CREATE TABLE CUSTOMER(  
    cno INT PRIMARY KEY,  
    cname VARCHAR(100),  
    c_type CHAR(1) CHECK (c_type IN ('A', 'B'))  
);  
  
CREATE TABLE ACCOUNTS(  
    ac_no INT PRIMARY KEY,  
    br_no INT REFERENCES BRANCHES(br_no),  
    cust_no INT REFERENCES CUSTOMER(cno),  
    ac_type VARCHAR(20),  
    bal NUMERIC(12,2)  
);  
|
```

```

-- a) Function to update customer type based on threshold
CREATE OR REPLACE FUNCTION update_customer_type(
    threshold_value NUMERIC(12,2),
    customer_number CUSTOMER.cno%TYPE
)
RETURNS VOID AS
$$
DECLARE
    total_balance NUMERIC(12,2);
BEGIN
    SELECT COALESCE(SUM(bal), 0) INTO total_balance FROM ACCOUNTS WHERE cust_no = customer_number;

    IF total_balance > threshold_value THEN
        UPDATE CUSTOMER SET c_type = 'A' WHERE cno = customer_number;
    ELSE
        UPDATE CUSTOMER SET c_type = 'B' WHERE cno = customer_number;
    END IF;
END;
$$
LANGUAGE 'plpgsql';

```

```

-- b) Function to close a branch and transfer accounts to another branch
CREATE OR REPLACE FUNCTION CloseBranch(
    closing_branch INT,
    takeover_branch INT
)
RETURNS VOID AS
$$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM BRANCHES WHERE br_no = takeover_branch) THEN
        RAISE EXCEPTION 'Takeover branch % does not exist', takeover_branch;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM BRANCHES WHERE br_no = closing_branch) THEN
        RAISE EXCEPTION 'Closing branch % does not exist', closing_branch;
    END IF;

    UPDATE ACCOUNTS SET br_no = takeover_branch WHERE br_no = closing_branch;

    DELETE FROM BRANCHES WHERE br_no = closing_branch;
END;
$$
LANGUAGE 'plpgsql';

```

```

--c--
CREATE OR REPLACE FUNCTION safe_withdrawal(
    account_number ACCOUNTS.ac_no%TYPE,
    withdrawal_amount NUMERIC(12,2)
)
RETURNS VARCHAR AS
$$
DECLARE
    current_balance NUMERIC(12,2);
BEGIN
    SELECT bal INTO current_balance FROM ACCOUNTS WHERE ac_no = account_number;

    IF current_balance IS NULL THEN
        RETURN 'ERROR: Account ' || account_number || ' does not exist';
    END IF;

    IF withdrawal_amount <= 0 THEN
        RETURN 'ERROR: Withdrawal amount must be positive';
    END IF;

    IF current_balance >= withdrawal_amount THEN
        UPDATE ACCOUNTS SET bal = bal - withdrawal_amount WHERE ac_no = account_number;
    END IF;
END;
$$
LANGUAGE 'plpgsql';

```
